

Remove Duplicates from a Sorted Doubly Linked List

Personal DSA Notes

The key property used here is that the doubly linked list is sorted, so equal values occur consecutively. The algorithm exploits that continuity instead of searching the whole list for every value.

■filecite■turn28file0■L58-L76■

1. Problem Statement

Given the head of a **sorted doubly linked list**, remove all duplicate nodes while keeping the first occurrence of each value. Return the head of the updated list. The head does not change because the first occurrence is retained. ■filecite■turn28file0■L18-L56■

2. Main Intuition

Because the list is sorted, duplicates of the current node can only appear immediately after it. Therefore, for each **temp**, use another pointer **nextNode** to skip every consecutive node having the same value. When the first different value is found, connect temp directly to that node.

Example:

```
1 <-> 1 <-> 1 <-> 2 <-> 3 <-> 3 <-> 4
^
temp
```

```
nextNode skips:
    1 -> 1 -> 2
```

Then connect:

```
temp.next = nextNode
nextNode.prev = temp
```

Result:

```
1 <-> 2 <-> 3 <-> 4
```

3. Why Sorting Is Important

If the list is sorted, equal values are contiguous. For example, all 1s are together and all 3s are together. A value cannot disappear and then appear again later. This lets the inner pointer skip duplicates once and never revisit them. ■filecite■turn28file0■L58-L94■

For an unsorted list such as 1 <-> 2 <-> 1, this exact approach would not remove both 1s.

4. Pointer Roles

```
temp      = node whose duplicates we are removing
nextNode  = moves forward to find the first different value
```

Example:

```
1 <-> 1 <-> 1 <-> 2
^
temp

      ^       ^       ^
duplicate duplicate first different
                nextNode moves here
```

5. Algorithm

1. Start temp at head.
2. Continue while temp has a next node.
3. Set nextNode = temp.next.
4. Move nextNode forward while it exists and its value equals temp.val.
5. Connect temp.next to nextNode.
6. If nextNode exists, set nextNode.prev = temp.
7. Move temp = temp.next.
8. Return head.

6. Pseudocode

```
temp = head

while temp != null AND temp.next != null:

    nextNode = temp.next

    while nextNode != null AND nextNode.val == temp.val:
        nextNode = nextNode.next

    temp.next = nextNode

    if nextNode != null:
        nextNode.prev = temp

    temp = temp.next

return head
```

7. Complete Java Code

```
class Solution {

    public ListNode removeDuplicates(ListNode head) {

        // temp keeps the first occurrence of each value.
        ListNode temp = head;

        while (temp != null && temp.next != null) {

            // Start from the node immediately after temp
            // and skip all consecutive duplicates.
            ListNode nextNode = temp.next;

            while (nextNode != null && nextNode.val == temp.val) {
                nextNode = nextNode.next;
            }

            // Connect the first occurrence directly
            // to the first node with a different value.
            temp.next = nextNode;

            // Repair the backward link if a next node exists.
            if (nextNode != null) {
                nextNode.prev = temp;
            }

            // Continue from the next remaining node.
            temp = temp.next;
        }

        return head;
    }
}
```

```
}
```

8. Comments — Why These Lines Matter

temp = head; — keeps the traversal pointer separate from head, because the head is returned at the end.

nextNode = temp.next; — starts checking for duplicates immediately after the first occurrence.

while (nextNode != null && nextNode.val == temp.val) — skips all consecutive duplicates. The null check must come first so nextNode.val is never accessed when nextNode is null.

temp.next = nextNode; — removes all skipped duplicate nodes from the forward chain.

nextNode.prev = temp; — repairs the backward link so the DLL remains valid.

temp = temp.next; — moves to the next distinct value after the links have been updated.

9. Why No Head Update?

The head is the first occurrence of the smallest value in a sorted list. The algorithm keeps that first occurrence and removes only nodes after it, so the head remains unchanged. The transcript explicitly notes that the head does not change. ■filecite■turn28file0■L50-L56■

10. Dry Run

Input:

1 <-> 1 <-> 1 <-> 2 <-> 3 <-> 3 <-> 4

Step 1:

temp = first 1

nextNode skips the other 1s

nextNode stops at 2

Set:

1.next = 2

2.prev = 1

List:

1 <-> 2 <-> 3 <-> 3 <-> 4

Step 2:

temp = 2

nextNode = 3

3 != 2, so nothing is skipped

Move temp to 3.

Step 3:

temp = 3

nextNode skips the second 3

nextNode stops at 4

Set:

3.next = 4

4.prev = 3

Final:

1 <-> 2 <-> 3 <-> 4

11. What Happens When Duplicates Are at the Tail?

1 <-> 2 <-> 3 <-> 3 <-> 3

^
temp

nextNode skips all duplicate 3s
and finally becomes null.

temp.next = null

Final:

1 <-> 2 <-> 3

When nextNode becomes null, there is no node whose prev pointer needs updating, so the if (nextNode != null) check prevents invalid access. The transcript specifically highlights this boundary case.

■filecite■turn28file0■L154-L170■

12. Important Traversal Insight

At first glance, the code has an outer loop and an inner loop, which may look like $O(n^2)$. But the inner loop does not repeatedly scan the same duplicate nodes after they have been bypassed. Once the links are changed, the outer pointer jumps to the first node that was not a duplicate. Thus, across the entire algorithm, each node is traversed only a constant number of times. ■filecite■turn28file0■L440-L526■

13. Time Complexity — Clear Explanation

Time Complexity: $O(n)$

Let n be the number of nodes. The outer pointer visits the remaining distinct nodes, while nextNode skips duplicate nodes. A duplicate node skipped by the inner loop is not later revisited by the outer loop because temp.next is changed to jump over it. Therefore, the total number of pointer movements across both loops is proportional to n , giving $O(n)$ time. The transcript explicitly explains why this is not $O(n^2)$.

■filecite■turn28file0■L444-L528■

14. Space Complexity — Clear Explanation

Extra Space: $O(1)$

Only two traversal pointers, temp and nextNode, are used. No array, HashMap, recursion stack, or other data structure is created. Therefore the extra space does not grow with n : $O(1)$.

■filecite■turn28file0■L528-L534■

15. Complexity Summary

Time: $O(n)$
Every node is traversed only a constant number of times.

Space: $O(1)$
Only temp and nextNode are used.

The solution is optimal for this problem.

16. Edge Cases

- Empty list → return null.
- One node → nothing to remove.
- No duplicates → links remain effectively unchanged.
- All nodes have the same value → only the first node remains and its next becomes null.
- Duplicates in the middle → bypass them and repair both links.
- Duplicates at the tail → nextNode becomes null and temp.next becomes null.
- Multiple groups of duplicates → each group is handled independently.

17. Quick Revision

Sorted DLL duplicate removal:

```
temp = head
```

```
while temp != null && temp.next != null:
```

```
    nextNode = temp.next
```

```
    while nextNode != null
```

```
        && nextNode.val == temp.val:
```

```
        nextNode = nextNode.next
```

```
    temp.next = nextNode
```

```
    if nextNode != null:
```

```
        nextNode.prev = temp
```

```
    temp = temp.next
```

```
return head
```

Remember:

SORTED -> DUPLICATES ARE CONTIGUOUS

SKIP DUPLICATES -> CONNECT TO FIRST DIFFERENT NODE

REPAIR BOTH NEXT AND PREV

TIME $O(n)$, EXTRA SPACE $O(1)$